

WEEK2 relational-algebra-notes-part2

Relational Algebra — Part 2: Set Operators, Renaming & Notations

Study notes based on the Stanford relational algebra lecture (Video 2 of 2). Builds on Part 1 (select, project, cross-product, natural join, theta join). Covers: the three **set operators** (union, difference, intersection), the **rename** operator, and **alternative notations** (assignment statements & expression trees).

0. Quick Recap from Part 1

A relational algebra expression is applied to a **set of relations** and produces a **relation** as its result. The same college-admissions database is used:

- **College** — cName (*key*), state, enrollment
- **Student** — sID (*key*), sName, GPA, sizeHS
- **Apply** — sID, cName, major (*composite key*), decision

Sample data (same as Part 1)

College

cName	state	enrollment
Stanford	CA	21000
Berkeley	CA	36000
MIT	MA	11000
Cornell	NY	21000
Harvard	MA	22000

Student

sID	sName	GPA	sizeHS
123	Amy	3.9	800

sID	sName	GPA	sizeHS
234	Bob	3.6	1500
345	Craig	3.5	500
456	Doris	3.9	1000
567	Edward	2.9	2000
678	Fay	3.8	200
789	Gary	3.4	800

Apply

sID	cName	major	decision
123	Stanford	CS	Y
123	Stanford	EE	N
123	Berkeley	CS	Y
234	Berkeley	biology	N
234	Stanford	CS	R
345	MIT	bioengineering	Y
345	Cornell	bioengineering	N
345	Cornell	CS	Y
345	Cornell	EE	R
456	Stanford	CS	N
678	Stanford	history	Y
789	Cornell	CS	R

So far we have: σ (select, rows), π (project, columns), $\times$ (cross-product), $\bowtie$ (natural join), $\bowtie_{\theta}$ (theta join).

1. The Key Distinction: Horizontal vs. Vertical Combining

Before the set operators, internalize this contrast — it explains *why* set operators exist:

Combining style	Operators	What it does
Horizontal	$\times$, $\bowtie$, $\bowtie_{\theta}$	Matches a tuple T1 from one relation

Combining style	Operators	What it does
		with a tuple T2 from another, producing wider rows (more columns).
Vertical	$\cup, -, \cap$	Stacks tuples from two relations into one combined list (same columns, more/fewer rows).

The joins from Part 1 combine **horizontally**. The set operators combine **vertically**.

Abstract picture

Two single-column relations $R_1(A)$ and $R_2(A)$:

R_1	R_2
A	A
a_1	a_2
a_2	a_3

Operator	Result table
$R_1 \cup R_2$	$\{a_1, a_2, a_3\}$
$R_1 - R_2$	$\{a_1\}$
$R_1 \cap R_2$	$\{a_2\}$

2. Set Operators — Schema Requirement

All three set operators (**union**, **difference**, **intersection**) require that **both sides have the same schema** — i.e., the **same attribute names** (and compatible types).

When the schemas don't naturally match, we fix them with the **rename operator** (Section 6). The lecture introduces the set operators first, then circles back to rename.

3. Union — $\cup$

Standard set union (as learned in elementary school): combine the tuples of two relations into one, **eliminating duplicates** (set semantics).

Example — list every college name *and* student name

We want one flat list. A join/cross-product is **wrong** here — those combine horizontally. We want a **vertical** list, so we use union:

$\pi_{\text{cName}}(\text{College}) \cup \pi_{\text{sName}}(\text{Student})$

1. $\pi_{\text{cName}}(\text{College}) \rightarrow$ list of all college names.
2. $\pi_{\text{sName}}(\text{Student}) \rightarrow$ list of all student names.
3. $\cup$ stacks them into a single column.

Inputs:

$\pi_{\text{cName}}(\text{College})$	$\pi_{\text{sName}}(\text{Student})$
Stanford	Amy
Berkeley	Bob
MIT	Craig
Cornell	Doris
Harvard	Edward
	Fay
	Gary

Result ($\cup$): 12 distinct names in one column.

name
Stanford
Berkeley
MIT
Cornell
Harvard
Amy
Bob
Craig
Doris

name

Edward

Fay

Gary

Technical caveat: For the union to be *well-formed*, both sides must share the same schema (attribute name). $cName \neq sName$, so strictly this needs a **rename** first — fixed in §6.1. The output column above is called name once both sides are renamed.

4. Difference — $-$ (extremely useful)

$E1 - E2$ = all tuples in $E1$ that are **not** in $E2$. Written with the **minus sign**. This is the operator that lets you express "... **that did NOT** ..." queries.

Example A — IDs of students who applied **nowhere**

Sounds complex; it's simple:

$\pi_{(sID)}(\text{Student}) - \pi_{(sID)}(\text{Apply})$

Inputs:

$\pi_{(sID)}(\text{Student})$	$\pi_{(sID)}(\text{Apply})$
123	123
234	234
345	345
456	456
567	678
678	789
789	

Result: the only student whose ID isn't in Apply is **567 (Edward)**.

sID

567

Example B — *names* of students who applied nowhere (the "join back" trick)

You **cannot** just add *sName* to both projections — *sName* doesn't exist in *Apply*, so the two sides would have mismatched schemas (one has (*sID*, *sName*), the other only *sID*). Subtraction requires matching schemas.

The trick: compute the IDs first, then *join back* to *Student* to recover the names.

```

$$\pi_{(sName)} ( \quad ( \pi_{(sID)}(Student) - \pi_{(sID)}(Apply) ) \bowtie Student \quad )$$

```

Step by step:

Step 1 — difference $\rightarrow$ just {567} (the table above).

Step 2 — $\dots \bowtie Student$ — natural join on *sID* re-attaches the full *Student* row:

sID	sName	GPA	sizeHS
567	Edward	2.9	2000

Step 3 — $\pi_{(sName)}(\dots)$:

sName
Edward

Why "join back" works: The natural join matches the small ID set against *Student* on the shared *sID*, giving back the rich schema of *Student* for exactly those students. This pattern — *filter down to keys, then join back to a full relation* — is very common and worth memorizing.

5. Intersection — $\cap$

$E1 \cap E2$ = tuples present in **both** *E1* and *E2*. Again, both sides must share the same schema.

Example — names that are both a college name and a student name

For this example, imagine our data also contained a college named *Washington* (say, (*Washington*, *WA*, 30000)) **and** a student named *Washington* (say, (890, *Washington*, 3.0, 1500)). The query:

$\pi_{\text{cName}}(\text{College}) \cap \pi_{\text{sName}}(\text{Student})$

Inputs (with the hypothetical Washington rows added):

$\pi_{\text{cName}}(\text{College})$	x	$\pi_{\text{sName}}(\text{Student})$
Stanford		Amy
Berkeley		Bob
MIT		Craig
Cornell		Doris
Harvard		Edward
Washington		Fay
		Gary
		Washington

Result: the only name appearing on both sides.

name
Washington

(With our original data — no shared names — the result would be empty.)

(Schemas must match — handled via rename, §6.1.)

Intersection adds NO expressive power — *two* proofs

Proof 1 — via difference:

$$E1 \cap E2 \equiv E1 - (E1 - E2)$$

Venn-diagram intuition: shade all of E1 (purple). Then $E1 - E2$ is the part of E1 outside E2 (green). Subtracting the green from the purple leaves exactly the overlap $E1 \cap E2$. So anything written with $\cap$ can be rewritten with $-$.

Proof 2 — via natural join (when schemas are identical):

$$E1 \cap E2 \equiv E1 \bowtie E2 \quad (\text{when } E1 \text{ and } E2 \text{ have the same schema})$$

Why: with identical schemas, natural join enforces equality on **every** column and drops duplicates — so only tuples appearing in both survive. That's exactly intersection.

Takeaway: $\cap$ is a *convenience*, not a necessity — but it's still very readable in queries.

6. Rename Operator — ρ (rho)

The **rename** operator **reassigns the schema** of an expression's result. Symbol: ρ (Greek rho). Like all operators, it applies to the result of **any** expression.

General form

$\rho_{\mathbf{R}}(\mathbf{A1}, \mathbf{A2}, \dots, \mathbf{An}) (\mathbf{E})$

"Compute E, then call the result relation **R** with attributes **A1...An**." This new schema is then usable in the enclosing expression.

Abbreviated forms

- **Rename relation only** (keep E's attribute names): $\rho_{\mathbf{R}} (\mathbf{E})$
- **Rename attributes only** (keep relation name): $\rho_{\mathbf{A1}, \dots, \mathbf{An}} (\mathbf{E})$ — must list attributes, otherwise it's indistinguishable from the relation-only form.

The rename operator solves **two distinct problems**:

6.1 Use #1 — Fixing schemas for set operators

Recall the union of college and student names had mismatched attribute names. Rename both sides to a common schema:

$$\begin{array}{c} \rho_{\mathbf{C(name)}} (\pi_{\mathbf{(cName)}}(\mathbf{College})) \\ \cup \\ \rho_{\mathbf{C(name)}} (\pi_{\mathbf{(sName)}}(\mathbf{Student})) \end{array}$$

Before rename — incompatible schemas:

$\pi_{\mathbf{(cName)}}(\mathbf{College})$	$\pi_{\mathbf{(sName)}}(\mathbf{Student})$
cName	sName
Stanford	Amy
Berkeley	Bob
...	...

After rename — both are now relation C(name):

$\rho_{\mathbf{C(name)}} (\pi_{\mathbf{(cName)}}(\mathbf{College}))$	$\rho_{\mathbf{C(name)}} (\pi_{\mathbf{(sName)}}(\mathbf{Student}))$
name	name

$\rho_{\text{C(name)}}(\pi_{\text{cName}}(\text{College}))$	$\rho_{\text{C(name)}}(\pi_{\text{sName}}(\text{Student}))$
Stanford	Amy
Berkeley	Bob
...	...

The union is now well-formed. This is purely a **syntactic necessity** for valid set operations.

6.2 Use #2 — Disambiguation in **self-joins** (the important use)

A **self-join** combines a relation **with itself**. You need rename so the two copies have **distinct names** to refer to.

7. Worked Example: Pairs of Colleges in the Same State (Self-Join)

Goal: find pairs of colleges located in the same state (e.g., *Stanford & Berkeley*).

This is **horizontal** combining (pairing rows), using **two instances of College**. But you can't write `state = state` — *which* state? The two copies have no distinct names. **Rename fixes this.**

Approach A — cross-product + rename + select

Rename the two instances so their attributes are distinguishable:

```
C1 :=  $\rho_{\text{C1(N1, S1, E1)}}(\text{College})$   -- name1, state1, enr1
C2 :=  $\rho_{\text{C2(N2, S2, E2)}}(\text{College})$   -- name2, state2, enr2
```

C1 and **C2** (same data, distinct attribute names):

N1	S1	E1
Stanford	CA	21000
Berkeley	CA	36000
MIT	MA	11000
Cornell	NY	21000
Harvard	MA	22000

Then:

$\sigma_{(S1 = S2)} (C1 \times C2)$

$C1 \times C2$ produces $5 \times 5 = 25$ rows. Filtering with $S1 = S2$ keeps only same-state pairs. The 9 rows that survive ($CA \times CA = 2 \times 2 = 4$, $MA \times MA = 2 \times 2 = 4$, $NY \times NY = 1 \times 1 = 1$):

N1	S1	E1	N2	S2	E2
Stanford	CA	21000	Stanford	CA	21000
Stanford	CA	21000	Berkeley	CA	36000
Berkeley	CA	36000	Stanford	CA	21000
Berkeley	CA	36000	Berkeley	CA	36000
MIT	MA	11000	MIT	MA	11000
MIT	MA	11000	Harvard	MA	22000
Harvard	MA	22000	MIT	MA	11000
Harvard	MA	22000	Harvard	MA	22000
Cornell	NY	21000	Cornell	NY	21000

Approach B — make the join column share a name, then natural join (slicker)

If we rename **both** state attributes to the **same** name S (but keep distinct name/enrollment attributes), the natural join automatically enforces equality on S:

$C1 := \rho_{(C1(N1, S, E1))} (College)$

$C2 := \rho_{(C2(N2, S, E2))} (College)$

$C1 \bowtie C2$ -- auto-matches on shared attribute S

This gives the same 9 rows but with **one** S column (since natural join drops the duplicate).

Fixing the three quirks of a self-join

The naive pairing above has problems — fix them with selection conditions:

1. **Self-pairs** (Stanford–Stanford, Berkeley–Berkeley, etc.): a college pairs with itself (5 rows). → add $N1 \neq N2$.

After $N1 \neq N2$, 4 rows remain:

N1	S	E1	N2	E2
Stanford	CA	21000	Berkeley	36000
Berkeley	CA	36000	Stanford	21000
MIT	MA	11000	Harvard	22000
Harvard	MA	22000	MIT	11000

2. **Mirror duplicates** (Stanford–Berkeley *and* Berkeley–Stanford): each pair appears twice in reverse. → **clever fix**: replace $N1 \neq N2$ with $N1 < N2$. This keeps only one ordering of each pair, killing both the self-pairs *and* the mirror duplicates in one stroke.

Final query:

$\sigma_{(N1 < N2)} (C1 \bowtie C2)$

Result: Berkeley < Stanford alphabetically, Harvard < MIT alphabetically.

N1	S	E1	N2	E2
Berkeley	CA	36000	Stanford	21000
Harvard	MA	22000	MIT	11000

Two clean pairs of same-state colleges, with no duplicates and no self-pairings.

The point of this example: the **rename operator is absolutely necessary** here — there is no way to express a self-join without it. (Self-pairs and mirror-pairs handling is a bonus lesson.)

8. Alternative Notations

Beyond the standard inline form, two other notations are common.

8.1 Assignment statements (modular / linear)

Break a big expression into named intermediate steps using $:=$. Same same-state-colleges query:

$C1 := \rho_{(C1(N1, S, E1))} (\text{College})$

$C2 := \rho_{(C2(N2, S, E2))} (\text{College})$

$CP := C1 \bowtie C2$

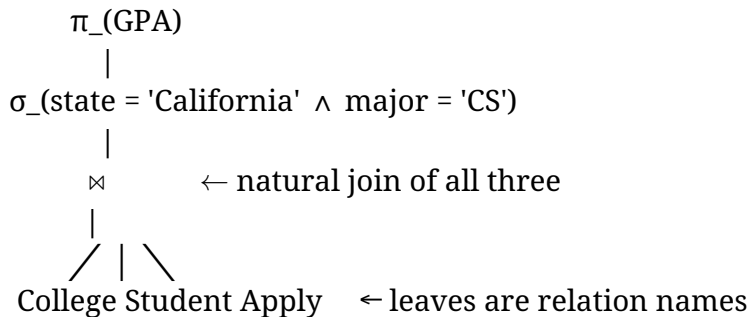
Answer $:= \sigma_{(N1 < N2)} (CP)$

Equivalent to the single big expression — just **modularized** for readability.
Useful for complex queries.

8.2 Expression trees (visualizing structure)

Operators become **internal nodes**; **relation names are always the leaves**.
Evaluation flows **bottom-up**.

Example: GPAs of students applying to CS in California (needs all three relations):



- **Leaves:** College, Student, Apply.
- The **natural join** of the three enforces $cName = cName$ (College Apply) and $sID = sID$ (Student Apply), yielding meaningful "student applies to college" triples.
- Then **select** on $state = 'California' \wedge major = 'CS'$, then **project** GPA.

Tracing through our data (state = CA, major = CS):

- CA colleges: Stanford (21000), Berkeley (36000).
- CS applications to those: (123, Stanford, CS, Y), (234, Stanford, CS, R), (456, Stanford, CS, N), (123, Berkeley, CS, Y).
- Joining with Student to get GPAs: Amy 3.9, Bob 3.6, Doris 3.9, Amy 3.9 (again).
- After projection + duplicate elimination:

GPA

3.9

3.6

Equivalent linear form:

$\pi_{\text{GPA}} (\sigma_{\text{state}='California' \wedge \text{major}='CS'} (\text{College} \bowtie \text{Student} \bowtie \text{Apply}))$

Why trees matter: They make an expression's structure visible, and — importantly — **when SQL is compiled inside a database system, it's**

typically turned into an expression tree very much like this (a query plan). This is the bridge from theory to real DBMS implementation.

9. The Core Language vs. Abbreviations

A major conceptual point: relational algebra has a small **core**, and everything else is **syntactic sugar** (abbreviations that add no expressive power).

Core constructs

Construct	Form	Role
Relation name	R	Base query
Select	$\sigma_c(E)$	Filter rows
Project	$\pi_{\text{attrs}}(E)$	Pick columns
Cross-product	$E1 \times E2$	Combine all tuple pairs
Union	$E1 \cup E2$	Vertical combine
Difference	$E1 - E2$	Set subtraction
Rename	$\rho_{(R(A...))}(E)$	Reassign schema

Abbreviations (rewritable using the core)

Abbreviation	Equivalent core expression
Natural join $E1 \bowtie E2$	$\pi_{\text{(schema union)}}(\sigma_{\text{(shared attrs equal)}}(E1 \times E2))$
Theta join $E1 \bowtie_{\theta} E2$	$\sigma_{\theta}(E1 \times E2)$
Intersection $E1 \cap E2$	$E1 - (E1 - E2)$ (or $E1 \bowtie E2$ when schemas match)

These abbreviations don't increase what the language *can* express — but they make queries far easier to read and write.

A note on parentheses

Parentheses are used purely for **disambiguation**, just like in arithmetic. With practice it's straightforward to tell when they're needed; you can omit them where precedence/associativity makes the meaning unambiguous (e.g., chained associative natural joins).

10. Summary

Relational algebra is a **formal language** based on **sets**. It uses **set operators** and **other operators that combine data from multiple relations**. It takes relations as input and produces relations as output, and it forms the **formal foundation of implemented relational database management systems**.

This video added:

- $\cup$ (**union**) and $-$ (**difference**) — *core*; vertical combining.
- $\cap$ (**intersection**) — *abbreviation*; provably reducible to $-$ or $\cup$ (with equal schemas) $\bowtie$.
- The "**join back**" pattern for recovering full rows after a key-level set operation.
- ρ (**rename**) — *core*; needed for set-operator schema matching and, crucially, for **self-joins**.
- The **same-state-colleges** self-join, including the $N1 < N2$ trick for self/mirror pairs.
- Two notations: **assignment statements** (modular) and **expression trees** (visual; mirror real SQL query plans).
- The big idea: a small **core** ($\sigma \pi \times \cup - \rho$ + relation names) plus **abbreviations** ($\bowtie \bowtie_{\theta} \cap$).

Quick symbol reference (Parts 1 & 2)

Symbol	Name	Meaning	Core or abbrev.
σ	sigma	select (rows)	core
π	pi	project (columns)	core
$\times$	times	cross-product	core
$\cup$	union	set union (vertical)	core
$-$	minus	set difference	core
ρ	rho	rename schema	core
$\bowtie$	bow tie	natural join	abbreviation
$\bowtie_{\theta}$	bow tie + θ	theta join	abbreviation

Symbol	Name	Meaning	Core or abbrev.
$\cap$	cap	intersection	abbreviation
$\wedge \vee \neg$	—	AND, OR, NOT	(conditions)

End of the two-part relational algebra series. You now have the full set of operators and the relationship between the minimal core and its convenient abbreviations.